Module 4: Portfolio Milestone

Start Assignment

- Points 50
- Submitting a file upload



Portfolio Milestone (50 points)

Important! Read First

Choose one of the following two assignments to complete this week. Do not do both assignments. Identify your assignment choice in the title of your submission. Review the **Module 8 Portfolio Project (https://csuglobal.instructure.com/courses/123286/assignments/2269095)** instructions.

Option #1: Using C Programming language

From the previous exercises, you have 2 files in your Linux installation: file1.txt and file2.txt, both with one million rows of random numbers. Create a new script, double_numbers.sh, to read each line of file1.txt, storing the number from each line into a variable, then write a value that is double the original number, into another file called newfile1.txt. Once this is done for the entire contents of file1.txt, display the time it took to run. Your script might look as follows:

```
#!/bin/bash

SECONDS=0

while read -r number

do

  let "number *= 2"

  echo $number >> newfile1.txt

done < file1.txt

duration=$SECONDS

echo $duration
```

You may want to create a smaller file to work with until the process is working correctly. You can use the "head" command to grab the first 10 rows of file1.txt into another file and use that file in your script:

head file1.txt > tempfile.txt

For proper syntax of reading files into variables, read this article: **How to read file line by line in Bash script (https://linuxhint.com/read_file_line_by_line_bash)** .

You can also use the "head" command to quickly verify that newfile1.txt contains numbers that are double the value of the corresponding rows of file1.txt:

head file1.txt newfile1.txt

Once you have the script working to your satisfaction, delete newfile1.txt and run double_numbers.sh. You should now have a file called newfile1.txt containing one million lines. You should also have information indicating how long this process took to run.

Using C, create a program to perform this task, with 3 methods of doing this:

1. Read the entire contents of file1.txt into memory, then process each row.
2. Read one row of file1.txt at a time and process it.
3. Split file1.txt into 2 parts and read each part into memory separately.

Be sure to capture execution times for each method. How do they compare to each other, and to that of the double_numbers.sh script? Were there any surprises for you, or did the results match your expectations? Describe your findings in detail.

Option #2: Using Python Programming Language

From the previous exercises, you have 2 files in your Linux installation: file1.txt and file2.txt, both with one million rows of random numbers. Create a new script, double_numbers.sh, to read each line of file1.txt, storing the number from each line into a variable, then write a value that is double the original number into another file called newfile1.txt. Once this is done for the entire contents of file1.txt, display the time it took to run. Your script might look as follows:

```
#!/bin/bash

SECONDS=0

while read -r number

do

  let "number *= 2"

  echo $number >> newfile1.txt

done < file1.txt
```

duration=$SECONDS

echo $duration

You may want to create a smaller file to work with until the process is working correctly. You can use the "head" command to grab the first 10 rows of file1.txt into another file, and use that file in your script:

head file1.txt > tempfile.txt

For proper syntax of reading files into variables, read this: **How to read file line by line in Bash script (https://linuxhint.com/read_file_line_by_line_bash)** .

You can also use the "head" command to quickly verify that newfile1.txt contains numbers that are double the value of the corresponding rows of file1.txt:

head file1.txt newfile1.txt

Once you have the script working to your satisfaction, delete newfile1.txt and run double_numbers.sh. You should now have a file called newfile1.txt containing one million lines. You should also have information indicating how long this process took to run.

Using Python, create a program to perform this task, with 3 methods of doing this:

1. Read the entire contents of file1.txt into memory, then process each row.
2. Read one row of file1.txt at a time and process it.
3. Split file1.txt into 2 parts and read each part into memory separately.

Be sure to capture execution times for each method. How do they compare to each other, and to that of the double_numbers.sh script? Were there any surprises for you, or did the results match your expectations? Describe your findings in detail.

CSC507_M04_PM

Criteria	Ratings				Pts
Program Functionality and Accuracy	**15 pts** **Meets Expectation** Program correctly reads file1.txt, doubles each number, and writes accurate results to newfile1.txt in all three methods. Output matches expected results.	**12 pts** **Approaches Expectation** Program runs but may contain minor logic or output errors in one method. Results mostly correct.	**9 pts** **Below Expectation** Program has significant logic errors or fails for multiple methods. Output unreliable or incomplete.	**6 pts** **Limited Evidence** Program fails to execute properly or produces no meaningful output.	15 pts
Implementation of Three Processing Methods	**10 pts** **Meets Expectation** All three methods (full read, line-by-line, split read) are implemented, function independently, and demonstrate correct file handling and logic.	**8 pts** **Approaches Expectation** Two methods are correctly implemented; third partially functional or missing a key feature.	**6 pts** **Below Expectation** Only one method implemented correctly, or multiple methods missing essential functionality.	**4 pts** **Limited Evidence** Incomplete or missing all required methods.	10 pts
Execution Time Measurement and Comparison	**10 pts** **Meets Expectation** Execution times are accurately captured for all methods and compared clearly with Bash script timing. Results are presented and explained thoughtfully.	**8 pts** **Approaches Expectation** Execution times collected for most methods; some inconsistencies or incomplete comparison to Bash script.	**6 pts** **Below Expectation** Limited or unclear timing data; comparisons vague or missing key observations.	**4 pts** **Limited Evidence** No timing data or analysis provided.	10 pts

Criteria	Ratings				Pts
Analysis and Discussion of Results	**10 pts** **Meets Expectation** Provides a detailed, insightful discussion of performance differences and expected vs. actual outcomes. Demonstrates understanding of memory and I/O efficiency.	**8 pts** **Approaches Expectation** Provides some discussion of results but lacks depth or clarity; touches on performance differences.	**6 pts** **Below Expectation** Minimal or generic discussion with little technical insight or connection to results.	**4 pts** **Limited Evidence** No analysis or explanation provided.	10 pts
Code Quality and Documentation	**5 pts** **Meets Expectation** Code is well-structured, readable, and includes meaningful comments explaining logic, functions, and methods. Follows good coding and naming conventions.	**4 pts** **Approaches Expectation** Code mostly readable; some comments missing or inconsistent formatting.	**3 pts** **Below Expectation** Code difficult to follow; lacks consistent style or documentation.	**2 pts** **Limited Evidence** Code is poorly written or undocumented.	5 pts
					Total Points: 50